

0.67.3. Les variables de la lettre dans leur propre fichier — en simple

Le 0.67 en simple : `variables.h` ne garde que `x` et `y` ; `principal.cpp`, une ligne dans la boucle.

1. Ce qu'il faut comprendre

C'est le 0.67, en plus simple : la boucle de la croix est remplacée par une ligne, `deplace_croix`.

`variables.h` rétrécit aussi : plus besoin d'`attente`, la console la tient elle-même. Il ne reste que `x` et `y`, la place de la lettre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1181 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          73 o  ← la console, à l'allumage - la mise en route de la console
deplace_croix()    217 o  ← main() - une fonction fournie par le langage
main()             32 o  ← démarrage - ta fonction
VBlank             8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage      28 o  ← image() - attend l'image suivante
AttendreVBlank     16 o  ← démarrage - attend que l'écran soit libre
AttendreAcces      12 o  ← EffacerCases - attend de pouvoir écrire une case
AdresseCase        10 o  ← effacer(), poser() - trouve l'adresse d'une case de l'écran
EffacerCases       12 o  ← effacer() - efface des cases
CopierTuiles       18 o  ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o  ← démarrage - vide les deux cartes de l'écran
LireManette        37 o  ← bouton() - lit les huit boutons
Tuiles             704 o  ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
bouton()           x 4    20 o sur place, dans deplace_croix() + LireManette (37 o)
effacer()           x 1    15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
poser()             x 1    13 o sur place, dans deplace_croix() + AdresseCase (10 o)
image()             x 1     3 o sur place, dans main() + AttendreImage (28 o)
images()            x 1     2 o sur place, dans deplace_croix()
```

Tes fonctions appelées :

```
deplace_croix() x 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// x et y : voir l'onglet variables.h

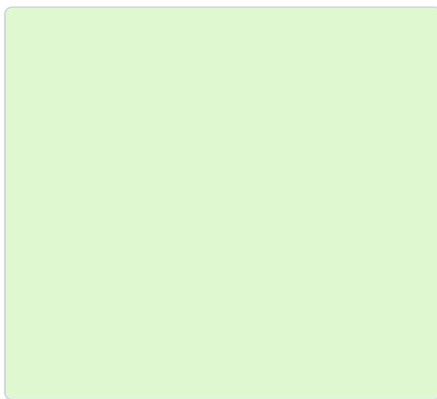
#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

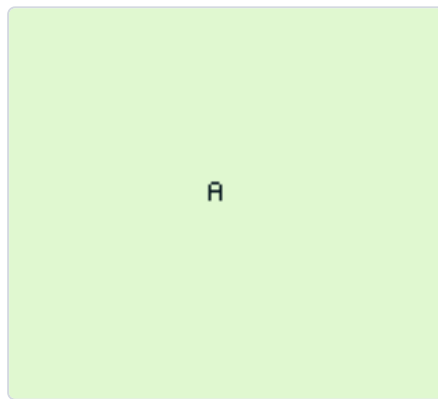
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.67 : un A qui se déplace avec les flèches, programme en deux onglets.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la croix le déplace

✓ il est à sa place